

# How to Retrieve Data from a Thread Function in C#

In this article, I am going to discuss **How to Retrieve Data from a Thread Function in C# using the Callback Method** with Examples. Please read our previous article before proceeding to this article where we discussed [How to Pass Data to Thread Function in a Type-Safe Manner in C#](#) with Examples. Here, we will see how to use a call-back method in C# to return data from a thread function. As part of this article, we will discuss the following pointers.

1. [What is a Callback Method in C#?](#)
2. [How does a callback method work in C#?](#)
3. [How to retrieve data from a thread function using a callback method in C#?](#)

## How to Retrieve Data from a Thread Function in C#?

As of now, we have discussed the uses of ThreadStart and ParameterizedThreadStart delegates. If you notice, the return type of these two delegates is void as shown in the below image.

```
public delegate void ThreadStart();  
public delegate void ParameterizedThreadStart(object obj);
```

So, using the above two delegates we cannot return any data from a method as the return type is void. Then the question that comes to our mind is how to retrieve the data from a thread function in C#? The answer is by using a callback method.

## How to Retrieve Data from a Thread Function using Callback Method in C#:

Let us see an example with step by step procedure to understand how we can use a callback method to retrieve the data from a thread function in C#. Please follow the below steps.

### Step1:

In order to retrieve the data from a thread function, first, you need to encapsulate the thread function and the data it requires in a helper class. To the constructor of the Helper class, you need to pass the required data if any as well as a delegate representing the callback method.

From the thread function body, you need to invoke the callback delegate just before the thread function ends. And one more thing you need to take care that the callback delegate and the actual callback method signature should be the same.

So, create a class file with the **NumberHelper.cs** and then copy and paste the following code into it. The code is explained through comments, so please go through the comment lines.

```
using System;  
namespace ThreadingDemo  
{
```

```

// First Create the callback delegate with the same signature of the call
public delegate void ResultCallbackDelegate(int Results);
//Creating the Helper class
public class NumberHelper
{
    //Creating two private variables to hold the Number and ResultCallback
    private int _Number;
    private ResultCallbackDelegate _resultCallbackDelegate;
    //Initializing the private variables through constructor
    //So while creating the instance you need to pass the value for Number
callback delegate
    public NumberHelper(int Number, ResultCallbackDelegate resultCallbackI
    {
        _Number = Number;
        _resultCallbackDelegate = resultCallbackDelagate;
    }
    //This is the Thread function which will calculate the sum of the num
    public void CalculateSum()
    {
        int Result = 0;
        for (int i = 1; i <= _Number; i++)
        {
            Result = Result + i;
        }
        //Before the end of the thread function call the callback method
        if (_resultCallbackDelegate != null)
        {
            _resultCallbackDelegate(Result);
        }
    }
}
}
}

```

## Step2:

In the second step, we need to create the callback method whose signature should be the same as the signature of the CallBack Delegate. In our example, ResultCallBackMethod is the callback method and its signature is the same as the signature of the ResultCallbackDelegate delegate. Within the Main method, we need to create an instance of the ResultCallbackDelegate delegate and while creating the instance we need to pass the ResultCallBackMethod as the parameter to its constructor. So when we invoke the delegate it will call the ResultCallBackMethod method.

Please modify the Program class code as shown below. The example code is self-explained. So, please go through the comment lines for a better understanding.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Create the ResultCallbackDelegate instance and to its constructor
        //pass the callback method name
        ResultCallbackDelegate resultCallbackDelegate = new
ResultCallbackDelegate(ResultCallBackMethod);
        int Number = 10;
        //Creating the instance of NumberHelper class by passing the Number
        //the callback delegate instance
        NumberHelper obj = new NumberHelper(Number, resultCallbackDelegate);
        //Creating the Thread using ThreadStart delegate
        Thread T1 = new Thread(new ThreadStart(obj.CalculateSum));

        T1.Start();
        Console.Read();
    }
    //Callback method and the signature should be the same as the callback
signature
    public static void ResultCallBackMethod(int Result)
    {
        Console.WriteLine("The Result is " + Result);
    }
}
}

```

Now run the application and you should see the output as expected.

Output: **The Result is 55**

## What is a Callback Function in C#?

We can define a callback function as a function pointer that is being passed as an argument to another function. And then it is expected to call back that function at some point in time.

In our example, we call the thread function of NumberHelper class from the Main method of the Program class. While creating the instance of NumberHelper class we pass the callback function as an argument to that class constructor. And then we expected that callback method to be called at some point in time by the NumberHelper method.

In the next article, I am going to discuss the [Join and IsAlive Methods of Thread Class in C#](#) with Examples. Here, in this article, I try to explain **How to Retrieve Data from a Thread Function in C#** using the Callback Method with Examples. I hope you understood this concept and enjoy How to Retrieve Data from a Thread Function in C# using the Callback Method with Examples article.